AI-Powered Phishing Detection System

High-Level Design and Used Tools

1. High-Level Design

1.1 System Overview

The objective of this system is to detect phishing emails using machine learning techniques. The system analyzes the content of an email and determines whether it is malicious (phishing) or legitimate (ham). It is designed as a modular pipeline where each component performs a specific task, ensuring clarity, scalability, and maintainability.

1.2 Architecture Description

The system follows a sequential pipeline architecture composed of multiple processing stages. Each stage transforms the input data and passes it to the next stage until a final classification is produced.

Pipeline Flow: Input → Preprocessing → Feature Extraction → Model → Output

1.3 System Components

1. Input Layer

The input layer is responsible for collecting email data from the user. This can be raw text pasted into the system or an uploaded email file. This layer ensures that the system can accept real-world email formats.

2. Preprocessing Module

This module prepares raw text for analysis by cleaning and normalizing it. Typical operations include converting text to lowercase, removing punctuation, removing stopwords, and tokenization. This step ensures that irrelevant noise is removed and the text is standardized for further processing.

3. Feature Extraction Module

The feature extraction module converts textual data into numerical form so that it can be used by machine learning algorithms. The primary method used is TF-IDF (Term Frequency–Inverse Document Frequency), which evaluates the importance of words in a document relative to a dataset. Additional features such as suspicious URLs, presence of urgent language, and sender inconsistencies can also be extracted.

4. Machine Learning Classification Module

This module applies supervised learning algorithms to classify emails. Common models include Logistic Regression and Naive Bayes. The model is trained on labeled datasets and learns patterns that distinguish phishing emails from legitimate ones. Once trained, it predicts the class of new incoming emails.

5. Output Layer

The output layer presents the classification result to the user. It indicates whether the email is phishing or legitimate. Optionally, the system may provide explanations such as key features that influenced the decision.

1.4 Functional Flow

1. The user inputs an email into the system. 2. The preprocessing module cleans and normalizes the text. 3. The feature extraction module converts text into numerical vectors. 4. The trained machine learning model analyzes the features. 5. The system outputs the final classification result.

1.5 Design Considerations

The system is designed to be modular, allowing each component to be improved independently. It is also scalable, meaning additional features or more advanced models can be integrated in the future. The focus is on explainability, ensuring that the predictions can be understood and justified.

2. Used Tools and Technologies

2.1 Programming Language

Python is used as the primary programming language due to its simplicity, readability, and strong ecosystem for data science and machine learning.

2.2 Machine Learning Libraries

Scikit-learn is used for implementing machine learning algorithms such as Logistic Regression and Naive Bayes. It also provides tools for feature extraction such as TF-IDF vectorization. It is chosen for its efficiency, ease of use, and suitability for text classification tasks.

2.3 Data Processing Libraries

Pandas is used for data manipulation, cleaning, and handling datasets. NumPy is used for numerical computations and efficient array operations.

2.4 Natural Language Processing Tools

NLTK or spaCy is used for text preprocessing tasks such as tokenization, stopword removal, and text normalization.

2.5 Web Framework (Optional)

Flask can be used to build a simple web interface that allows users to input emails and view predictions in real time.

2.6 Visualization Tools (Optional)

Matplotlib or Seaborn can be used to visualize model performance metrics such as accuracy, precision, recall, and confusion matrix.

3. Justification of Tool Selection

The selected tools provide a balance between performance and simplicity. Python ensures rapid development, Scikit-learn provides reliable models, and NLP libraries enable efficient text processing. Together, they allow the system to be implemented effectively while remaining easy to understand and maintain.

4. Conclusion

The proposed system provides a structured and efficient approach to detecting phishing emails using machine learning. The modular pipeline design ensures clarity and scalability, while the chosen tools support accurate and interpretable predictions. This design serves as a strong foundation for implementing a practical cybersecurity solution.